

Entregable 5. Seguridad

Controles del piloto y los que quedan como diseno para produccion. La columna Estado distingue lo verificado en ejecucion de lo propuesto.

5.1 Resumen

Control	Estado	Donde
Autenticacion	Implementado	JWT HS256, servicio de tokens
Autorizacion	Implementado	Segregacion por entidad en cada consulta
Almacenamiento de tokens	Con reserva	localStorage en el portal
Proteccion de APIs	Implementado	Authorize por defecto y validacion de modelo
CORS	Implementado	Lista blanca de origenes
CSP	Diseno	Corresponde al proxy inverso
Secretos	Implementado	Plantilla versionada, valores fuera del repositorio
Auditoria	Implementado	Tabla historial_estados y vista vw_trazabilidad
Rate limiting	Implementado	Ventana global y politica de login
Datos sensibles	Implementado	AES-GCM en NIT y cuenta bancaria

5.2 Autenticacion

JWT firmado con HS256, vigencia de 8 horas y ClockSkew reducido a 30 segundos. El valor por defecto de .NET son 5 minutos, que amplia la ventana en la que un token ya expirado sigue siendo aceptado.

Las contraseñas se almacenan con PasswordHasher de ASP.NET Identity, que aplica PBKDF2 con sal por usuario. En la base solo vive el hash.

Bloqueo de cuenta: 5 intentos fallidos bloquean 15 minutos, y el contador se reinicia al autenticar correctamente. Frena la fuerza bruta dirigida a un usuario concreto; el rate limiting frena la dirigida a muchos.

En produccion esto lo emitiría un servicio de identidad OIDC con MFA, no la propia API.

5.3 Autorizacion

El piloto no usa roles: el control es la **segregacion por entidad**, presente en la firma de cada metodo del repositorio.

```
Task<Solicitud?> ObtenerAsync(Guid id, long entidadId, CancellationToken ct = default);
```

No es un filtro opcional que se pueda olvidar: sin entidadId el codigo no compila. La entidad sale del claim del token, nunca de la peticion.

Verificado: un usuario de Gobernacion pidiendo una solicitud de Finanzas recibe **404**, no 403, de modo que no se confirma siquiera que el recurso exista.

En produccion se anadirian permisos por operacion, como politicas, sin tocar la segregacion por entidad, que es una capa distinta.

5.4 Almacenamiento de tokens

El portal guarda el token en localStorage. Es la decision mas discutible del piloto: sobrevive al cierre de pestana y es accesible desde JavaScript, de modo que un XSS lo expondria.

La alternativa correcta en produccion es una cookie HttpOnly, Secure y SameSite estricta, con token de refresco, que JavaScript no puede leer. Se eligio localStorage por simplicidad de demostracion, no porque sea lo adecuado.

5.5 Proteccion de APIs

Todos los controladores llevan Authorize; solo el login y el health check quedan abiertos. La validacion de modelo se hace con anotaciones y las reglas de negocio viven en el dominio.

El **manejo global de errores** traduce las excepciones a codigos HTTP estables (404, 403, 409, 422 y 500) con cuerpo application/problem+json. Un error 500 nunca devuelve la traza: devuelve el identificador de correlacion para el reporte.

HTTPS y HSTS los aporta el proxy inverso en produccion.

5.6 CORS

```
politica.WithOrigins(origenes).AllowAnyHeader().AllowAnyMethod()  
    .WithExposedHeaders("X-Correlation-Id")
```

Lista blanca desde configuracion, nunca AllowAnyOrigin. WithExposedHeaders es necesario para que el navegador deje al portal leer la cabecera de correlacion.

5.7 CSP

No implementada: la politica de seguridad de contenido se emite en el proxy inverso, que el piloto no incluye. La cabecera propuesta:

```
default-src 'self'; script-src 'self';  
style-src 'self' 'unsafe-inline';  
connect-src 'self' https://api.sigop.gob.gt;  
frame-ancestors 'none'
```

frame-ancestors none evita el secuestro por marco, y connect-src acota a donde puede llamar el portal.

5.8 Secretos

Los appsettings.json versionados son plantillas con marcadores de la forma __NOMBRE__. Los valores reales viven en appsettings.Development.json, excluido por .gitignore. El repositorio no contiene ninguna contrasena, clave ni cadena de conexi3n real, y la integraci3n continua lo comprueba en cada push.

En produccion los inyectaria un gestor de secretos como variables de entorno, con rotaci3n periodica. La plantilla ya deja preparado el punto de sustituci3n.

5.9 Auditoria

Cada cambio de estado deja una fila en historial_estados con estado anterior y nuevo, motivo, origen, actor, identificador de correlacion e instante. **No es opcional**: lo escribe el propio agregado dentro de la transacci3n, no un interceptor que se pueda omitir.

La vista vw_trazabilidad une esos cambios con los mensajes publicados:

```
SELECT * FROM vw_trazabilidad WHERE numero = 'SOL-2026-000009' ORDER BY momento;
```

5.10 Rate limiting

Dos ventanas fijas, particionadas por direccion IP:

Ambito	Limite	Contra que
Global	120 peticiones por minuto	Abuso general y saturacion
Endpoints de autenticacion	10 peticiones por minuto	Fuerza bruta de credenciales

Verificado: al intento 11 la API responde 429.

Se combina con el bloqueo de cuenta: el rate limiting frena a un atacante que prueba muchos usuarios desde una IP, y el bloqueo frena a quien insiste sobre un usuario desde muchas IP. En produccion el limite iria ademas en el gateway y por identidad, no solo por IP, porque detras de un NAT una IP agrupa a muchos usuarios legitimos.

5.11 Datos sensibles

El NIT del beneficiario y la cuenta bancaria se cifran con **AES-GCM de 256 bits** mediante un ValueConverter de EF Core: el cifrado es transparente al escribir y el descifrado al leer, y ninguna consulta puede olvidarlo.

Los valores llevan prefijo v1: para permitir rotacion de clave sin migracion masiva, y el conversor tolera valores sin prefijo. El campo se guarda en varchar de 500 porque el texto cifrado en base64 ocupa bastante mas que el original.

Para mostrar en pantalla existe enmascaramiento. Los cuerpos de los mensajes en RabbitMQ no llevan estos campos en claro: el consumidor los lee cifrados de la base y los descifra cuando los necesita.